Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Архитектура вычислительных систем»

ОТЧЕТ
к лабораторной работе №2
на тему:
«ИЗУЧЕНИЕ ЗАЩИЩЕННОГО РЕЖИМА РАБОТЫ ПРОЦЕССОРА PENTIUM»
БГУИР 6-05-0612-02 113

Выполнил студент группы 453503
ХАЛАМОВ Николай Андреевич

_____
(дата, подпись студента)

Проверил сотрудник каф. Информатики
МАКСИМЧИКОВ АРТЁМ ВЯЧЕСЛАВОВИЧ

_____
(дата, подпись преподавателя)

Минск 2026

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

Цель работы: изучить особенности защищенного режима процессора, получить практические навыки по программированию переключения процессора из реального режима в защищенный и обратно.

Написать программу, переключающую процессор в защищенный режим, выводящую на экран сообщение и затем возвращающую процессор в реальный режим. Вариант задания не предусмотрен.

2 ВЫПОЛНЕНИЕ РАБОТЫ

После включения питания процессор *Pentium* работает в реальном режиме (*Real Mode*), в котором он эмулирует работу процессора *Intel* 8086: физический адрес формируется как сумма смещения и значения сегментного регистра, сдвинутого на 4 разряда влево, поэтому доступно только 1 Мбайт памяти, а средства защиты отсутствуют. Защищенный режим (*Protected Mode*) позволяет адресовать до 4 Гбайт физической памяти и обеспечивает защиту сегментов с помощью уровней привилегий. В этом режиме сегментные регистры содержат не адрес сегмента, а селектор, указывающий на дескриптор в таблице дескрипторов.

Описание сегментов хранится в глобальной таблице дескрипторов *(GDT, Global Descriptor Table)*. Каждый дескриптор занимает 8 байт и содержит базовый линейный адрес сегмента, его предел (размер) и байт прав доступа. Положение таблицы в памяти задается регистром *GDTR* (6 байт: 16-разрядный предел и 32-разрядный линейный базовый адрес), который загружается командой *lgdt*. Для решения задачи была создана таблица из четырех дескрипторов.

Первый дескриптор с селектором *0x00* является нулевым (его база и предел равны 0, байт доступа *0x00*). Второй дескриптор с селектором *0x08* описывает сегмент кода с пределом *0xFFFF* и байтом доступа *0x9A*. Третий дескриптор с селектором *0x10* описывает сегмент данных и стека с пределом *0xFFFF* и байтом доступа *0x92*. Четвертый дескриптор с селектором *0x18* описывает видеопамять с базой *0xB8000*, пределом *0xFFFF* и байтом доступа *0x92*.

Байт доступа *0x9A* описывает присутствующий в памяти сегмент кода нулевого уровня привилегий, доступный для исполнения и чтения, а байт *0x92* – сегмент данных, доступный для чтения и записи. Так как программа имеет формат *COM* и адрес сегмента, в который она будет загружена, заранее неизвестен, базовые адреса сегментов кода и данных вычисляются во время работы: значение регистра *CS*, умноженное на 16, разбивается на части и записывается в соответствующие поля дескрипторов. Дескриптор видеопамяти имеет фиксированную базу *0xB8000*, которая является адресом текстового видеобуфера режима *VGA 80x25*.

Программа устанавливает текстовый видеорежим *80x25* и выводит стартовое сообщение средствами *DOS*, после чего сохраняет регистры стека и маски прерываний, динамически заполняя базовые адреса дескрипторов *GDT*. Далее запрещаются все прерывания, загружается регистр *GDTR* и устанавливается бит *PE* в *CR0*, а критический сброс очереди предвыборки команд и загрузка селектора кода в *CS* выполняются дальним переходом. В защищенном режиме в сегментные регистры загружаются селекторы данных и видеопамяти для прямого вывода сообщения в видеопамять с заданным атрибутом цвета. Для возврата в реальный режим в сегментные регистры

загружается селектор с пределом 64 Кбайт, сбрасывается бит *PE* и выполняется обратный дальний переход, по завершении которого восстанавливаются сохраненные регистры, стек и маски прерываний, выводится финальное сообщение и программа корректно завершается.

Программа написана на языке ассемблера *NASM* для формата *COM* и представлена на листинге 1. Переход в защищенный режим и возврат выполняются автоматически, без ожидания нажатия клавиши.

Листинг 1 – Программа переключения процессора в защищенный режим и обратно

```
[BITS 16]
[ORG 0x100]

section .text

start:
    mov ax, 0x0003
    int 10h
    mov dx, msg_real_start
    mov ah, 0x09
    int 21h

    mov [saved_ss], ss
    mov [saved_sp], sp
    movzx esp, sp

    mov ax, cs
    mov [patch_rm_seg], ax

    mov ax, cs
    movzx eax, ax
    shl eax, 4

    mov [gdt_code + 2], ax
    ror eax, 16
    mov [gdt_code + 4], al
    mov [gdt_code + 7], ah

    mov ax, cs
    movzx eax, ax
    shl eax, 4

    mov [gdt_data + 2], ax
    ror eax, 16
    mov [gdt_data + 4], al
    mov [gdt_data + 7], ah

    mov ax, cs
    movzx eax, ax
    shl eax, 4
    add eax, gdt_table
    mov [gdtr_addr], eax

    in al, 0x21
    mov [saved_pic1], al
    in al, 0xA1
    mov [saved_pic2], al
```

```
    cli
    in al, 0x70
    or al, 0x80
    out 0x70, al

    mov al, 0xFF
    out 0x21, al
    out 0xA1, al

    lgdt [gdtr]

    mov eax, cr0
    or al, 1
    mov cr0, eax


    jmp 0x08:pm_entry

pm_entry:
    mov ax, 0x10
    mov ds, ax
    mov es, ax
    mov ss, ax
    mov esp, stack_top

    mov ax, 0x18
    mov es, ax
    mov si, msg_pm
    mov di, (12 * 80 + 22) * 2
    mov ah, 0x1F

print_loop:
    lodsb
    test al, al
    jz print_done
    mov [es:di], ax
    add di, 2
    jmp print_loop

print_done:

    mov ax, 0x10
    mov ds, ax
    mov es, ax
    mov fs, ax
    mov gs, ax
    mov ss, ax

    mov eax, cr0
    and al, 0xFE
    mov cr0, eax

    db 0xEA
    dw rm_entry
patch_rm_seg:
    dw 0

rm_entry: ; r-mode
    mov ax, cs
    mov ds, ax
    mov es, ax
    mov ss, [saved_ss]
```

```
        movzx esp, word [saved_sp]

        mov al, [saved_pic1]
        out 0x21, al
        mov al, [saved_pic2]
        out 0xA1, al

        in al, 0x70
        and al, 0x7F
        out 0x70, al

        sti

        mov dx, msg_real_back
        mov ah, 0x09
        int 21h

        mov ax, 0x4C00
        int 21h
    saved_ss    dw 0
    saved_sp    dw 0
    saved_pic1  db 0
    saved_pic2  db 0

    msg_real_start db '1. [Real Mode] Transitioning to Protected Mode...',
13, 10, '$'
    msg_pm         db '>>> It is protected mode, goodbye! <<<', 0
    msg_real_back   db '2. [Real Mode] Successfully returned to DOS!', 13,
10, '$'

    stack_bottom:
        times 256 db 0
    stack_top:

    align 8
    gdtr:
        gdtr_limit dw gdt_end - gdt_table - 1
        gdtr_addr  dd 0

    gdt_table:
        dq 0

        gdt_code:
        dw 0xFFFF
        dw 0x0000
        db 0x00
        db 0x9A
        db 0x00
        db 0x00

        gdt_data:
        dw 0xFFFF
        dw 0x0000
        db 0x00
        db 0x92
        db 0x00
        db 0x00

        gdt_video:
        dw 0xFFFF
        dw 0x8000
        db 0x0B
```

```
    db 0x92
    db 0x00
    db 0x00

gdt_end:
```

Программа была собрана ассемблером *NASM* с помощью команды *nasm -f bin lab2.asm -o lab2.com*, а затем запущена в эмуляторе *DOSBox*. Результат корректной работы программы представлен на рисунке 1.

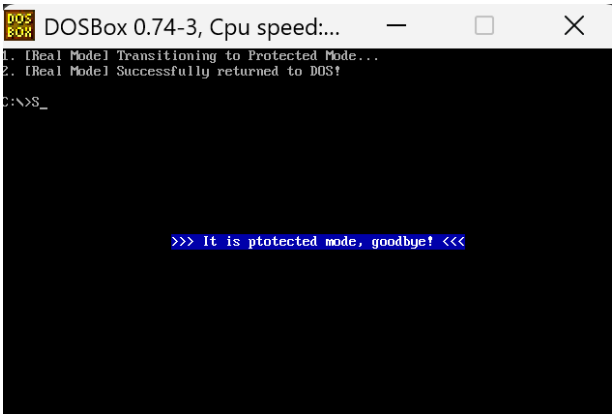


Рисунок 1 – Результат работы программы

Как видно из результатов выполнения программы, сообщение «1. [*Real Mode*] *Transitioning to Protected Mode...*» выведено в реальном режиме. Строка >>> *It is protected mode, goodbye!* <<< выведена в центре экрана уже в защищенном режиме путем записи в видеопамять, после чего процессор вернулся в реальный режим, и *DOS* вывела сообщение «2. [*Real Mode*] *Successfully returned to DOS*!».

В программе все прерывания на время работы в защищенном режиме запрещены, поскольку таблица дескрипторов прерываний (*IDT*) не создавалась. Расширение разрядности адреса (линия *A20*) включать не требуется, так как все используемые сегменты находятся в пределах первого мегабайта памяти.

ВЫВОД

В результате выполнения лабораторной работы были подробно изучены принципы функционирования защищенного режима процессора *Pentium*, включая механизмы сегментной адресации через селекторы и дескрипторы, организацию глобальной таблицы дескрипторов (*GDT*) и роль бита *PE* в регистре *CR0*. На практике были закреплены навыки программного переключения процессора между реальным и защищенным режимами.

Разработанный алгоритм успешно инициирует защищенный режим, осуществляет прямой вывод текстового сообщения в видеопамять *VGA* и выполняет корректный возврат в реальный режим для завершения программы в среде *DOS*. Ключевым практическим выводом стало понимание необходимости предварительной загрузки сегментных регистров дескрипторами с 64-килобайтным лимитом перед сбросом бита *PE*, а также обязательного восстановления контекста стека и масок прерываний после возвращения в реальный режим.